EZ Calendar and Weekly View

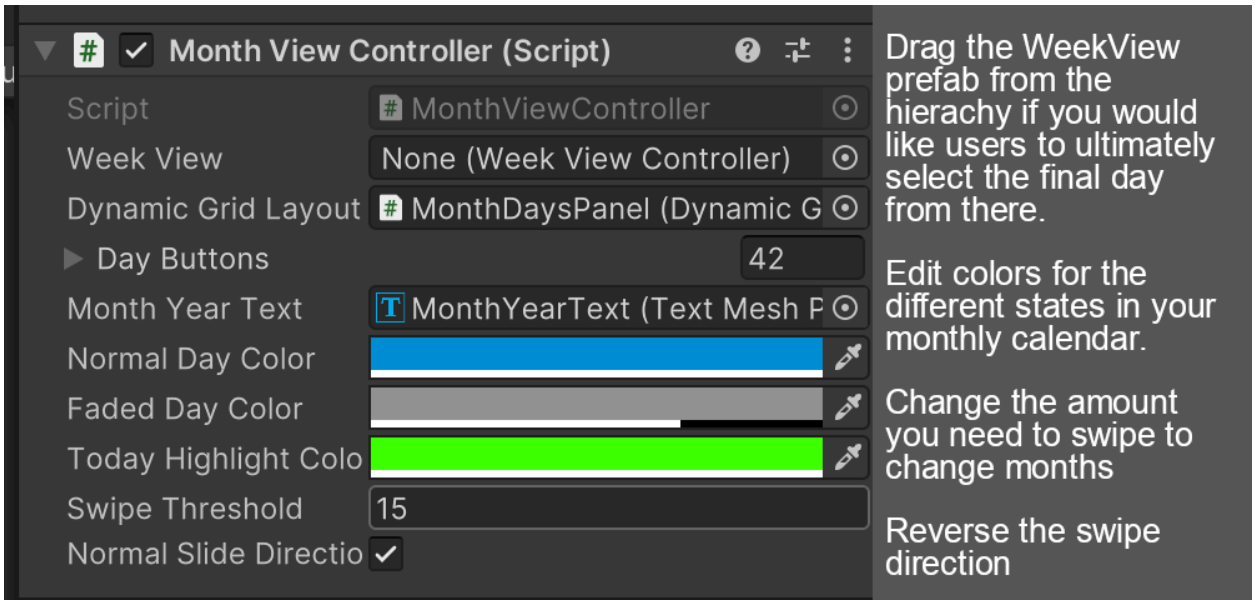
Overview

The EZ Calendar and Weekly View App provides a user-friendly interface for managing and viewing dates on a monthly and weekly basis. It includes two main views: the month view and the week view. Users can navigate through months and weeks via button clicks or swiping and create their own logic after selecting a date.

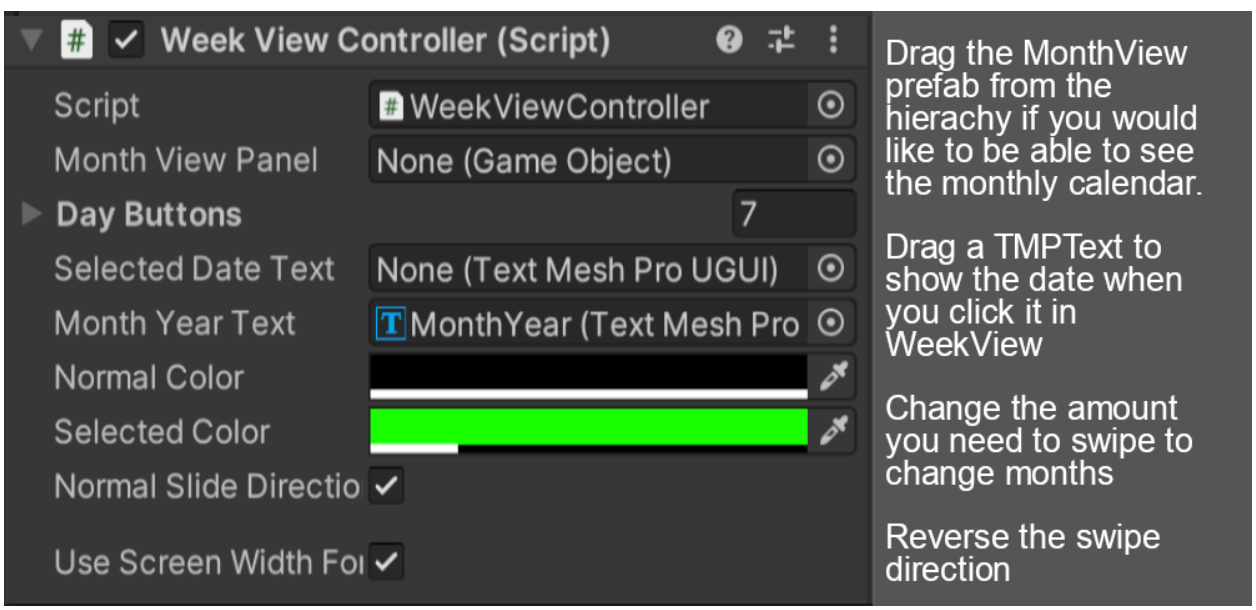
Quickstart Guide

1. **Import the EZ Calendar and Weekly View App Package:**
    - Import the provided Unity package into your project.
2. **Have a look at the example scenes.**
    - There are three example scenes.
        1. **FullDisplay**: This scene has a weekly view, a monthly view, and a panel below that shows the selected date and describes how one might want to integrate their logic into date selections. Write your custom logic in the `ClickedDateLogic` method.
        2. **JustWeekAndMonth**: This scene has a weekly and monthly view, and nothing else. Write your custom logic in the `ClickedDateLogic` method.
        3. **JustMonth**: This scene doesn't use the weekly view, instead, you select a date from the month and call logic straight from there. Write your custom logic in the `MonthOnlyCustomMethod` method.
3. **Set Up the Scene:**
    - Either duplicate and use the example scenes as a base for your application OR drag the MonthView AND/OR WeekView Prefabs into your existing scene as a child of the Canvas GameObject.
4. **Configure the Calendar App:**
    - Assign the necessary UI elements to the public variables in the `MonthViewController` and `WeekViewController` scripts using the Inspector.

MonthView



WeekView



5. **Run the Scene:**
    - Press Play and interact with the month calendar and/or weekly view.

NOTE: Don't Overcomplicate it. Feel free to change any logic as you feel fit, but if you want to use the functionality as is and have it interact with your code, you can simply edit the method designed for you to do just that.

- Write your custom logic in the `MonthOnlyCustomMethod` method in MonthViewController if you're only using the MonthView prefab with no WeekView attached in the inspector. This method gets kicked off from MonthView when you select a date (if you don't have WeekView attached in the inspector)
- Write your custom logic in the `ClickedDateLogic` method in WeekViewController if you're using the WeekView prefab in any way. This method gets kicked off from WeekView when you select a date

Editable Variables in the Inspector

WeekViewController

- **monthViewPanel**: Reference to the month view panel.
- **dayButtons**: List of buttons representing days in the week.
- **selectedDateText**: Text element for displaying the selected date.
- **monthYearText**: Text element for displaying the current month and year.
- **normalColor**: Color for normal days.
- **selectedColor**: Color for highlighting the selected day.
- **normalSlideDirection**: Determines the direction of slide navigation.
- **useScreenWidthForSwipe**: Determines if screen width is used for swipe threshold.
- **swipeThreshold**: Threshold for detecting swipe gestures (only visible if `useScreenWidthForSwipe` is false).

MonthViewController

- **weekView**: Reference to the `WeekViewController` instance.
- **dynamicGridLayout**: Reference to the `DynamicGridLayout` component for arranging day buttons.
- **dayButtons**: List of buttons representing days in the month.
- **monthYearText**: Text element for displaying the current month and year.
- **normalDayColor**: Color for normal days.
- **fadedDayColor**: Color for days outside the current month.
- **todayHighlightColor**: Color for highlighting the current day.
- **swipeThreshold**: Threshold for detecting swipe gestures.
- **normalSwipe**: Determines the direction of swipe navigation.

Classes

1. WeekViewController

Description

`WeekViewController` is responsible for managing the week view UI. It allows users to navigate between weeks, view days of the current week, and select specific dates. The class handles updating the UI elements and highlighting the selected date.

Public Variables

- `GameObject monthViewPanel` - Reference to the month view panel.
- `List<Button> dayButtons` - List of buttons representing days in the week.
- `TextMeshProUGUI selectedDateText` - Text element for displaying the selected date.
- `TextMeshProUGUI monthYearText` - Text element for displaying the current month and year.
- `Color normalColor` - Color for normal days.
- `Color selectedColor` - Color for highlighting the selected day.
- `bool normalSlideDirection` - Determines the direction of slide navigation.
- `float swipeThreshold` - Threshold for detecting swipe gestures.
- `bool useScreenWidthForSwipe` - Determines if screen width is used for swipe threshold.

Public Methods

- `void SetCurDate(DateTime date)` - Sets the current date and updates the UI.
- `void GoToPreviousWeek()` - Navigates to the previous week.
- `void GoToNextWeek()` - Navigates to the next week.
- `void GoToToday()` - Navigates to the current date.
- `void ShowMonthView()` - Displays the month view panel.
- `void HideMonthView()` - Hides the month view panel.
- `void GoToDate(DateTime date)` - Navigates to the specified date's week.

Private Methods

- `void Start()` - Initializes the week view and sets up event listeners.
- `void ClickedDateLogic(DateTime date)` - Custom logic for handling date clicks.
- `void OnDayButtonClick(int index)` - Handles day button clicks.
- `void UpdateSelectedDateText(DateTime date)` - Updates the selected date text.
- `DateTime GetStartOfWeek(DateTime date)` - Calculates the start of the week for a given date.
- `void UpdateWeekView()` - Updates the week view UI elements.
- `void HighlightSelectedButton()` - Highlights the selected day button.

- `void OnBeginDrag(PointerEventData eventData)` - Handles the beginning of a drag event.
- `void OnDrag(PointerEventData eventData)` - Handles drag events for swipe navigation.
- `void OnEndDrag(PointerEventData eventData)` - Handles the end of drag events for swipe navigation.
- `void UpdateWeekDatesDuringDrag()` - Updates week dates during drag to simulate swipe motion.
- `int CalculateDayShiftForSwipe(bool isEndDrag)` - Calculates day shift based on swipe distance.

---

2. MonthViewController

Description

`MonthViewController` is responsible for managing the month view UI. It allows users to navigate between months, view days of the current, previous, and next month, and select specific dates. The class handles updating the UI elements and highlighting the selected date.

Public Variables

- `WeekViewController weekView` - Reference to the week view controller.
- `DynamicGridLayout dynamicGridLayout` - Layout controller for arranging day buttons.
- `List<Button> dayButtons` - List of buttons representing days in the month.
- `TextMeshProUGUI monthYearText` - Text element for displaying the current month and year.
- `Color normalDayColor` - Color for normal days.
- `Color fadedDayColor` - Color for days outside the current month.
- `Color todayHighlightColor` - Color for highlighting the current day.
- `float swipeThreshold` - Threshold for detecting swipe gestures.
- `bool normalSwipe` - Determines the direction of swipe navigation.

Public Methods

- `void ShowMonthView(DateTime date)` - Displays the month view for the specified date.
- `void HideMonthView()` - Hides the month view.
- `void UpdateMonthView()` - Updates the month view UI elements.
- `void GoToPreviousMonth()` - Navigates to the previous month.

- `void GoToNextMonth()` - Navigates to the next month.
- `void GoToPreviousYear()` - Navigates to the previous year.
- `void GoToNextYear()` - Navigates to the next year.
- `void GoToToday()` - Navigates to the current date.
- `void OnDrag(PointerEventData eventData)` - Handles drag events for swipe navigation.
- `void OnEndDrag(PointerEventData eventData)` - Handles the end of drag events for swipe navigation.

Private Methods

- `void MonthOnlyCustomMethod(DateTime date)` - Custom logic for handling date clicks.
- `void UpdateButtonsForPreviousMonth(int daysInPreviousMonth, int startDayOfWeek)` - Updates buttons for the previous month.
- `void UpdateButtonsForCurrentMonth(int daysInMonth, int startDayOfWeek)` - Updates buttons for the current month.
- `void UpdateButtonsForNextMonth(int startDayOfWeek, int daysInMonth)` - Updates buttons for the next month.
- `void UpdateDayButton(Button button, DateTime date, Color color)` - Updates a day button with the specified date and color.
- `bool IsDateFromDifferentMonth(DateTime date)` - Checks if a date is from a different month.
- `void HighlightButton(Button button, Color color)` - Highlights a button with the specified color.
- `void OnDayButtonClick(DateTime date)` - Handles day button clicks.
- `void DeselectAllDayButtons()` - Deselects all day buttons.

Namespace: EZCalendarWeeklyView

All classes are organized under the `EZCalendarWeeklyView` namespace to maintain a clean and modular code structure that won't disturb your code in any way.

Need additional help?

Please send me a message or consult the youtube videos on the asset store page. I am happy to help!